

Transaction Chains in a Bookings Application

Omar Attaalla, Junko Kobayashi

December 2024

Abstract

In this paper we discuss an implementation of Transaction Chains to a bookings application. In geo-distributed systems, it is difficult to achieve serializability while still having low latency. Transaction Chains solves this issue through a mechanism of splitting transactions into hops leading to more serializable transactions. This is particularly useful in a bookings application because low latency is essential to the user's experience of the platform. We will also describe an implementation of transaction chains for this application as well as a solution to serializing dynamic transactions.

1 Introduction

Serializability is an essential part of database systems since it ensures a consistent output of transactions. However, in geo-distributed systems, achieving serializability and low latency is particularly challenging. One reason for this is in most normal database systems that guarantee serializability, transactions must run sequentially and completely for a return to happen to a user (and some transactions are not serializable). Since different database systems at different geo-locations require a different latency (based on the RTT and compute at each system), users will need to wait significant time to receive the value they require. This delay reduces the user experience significantly and can reduce the amount of revenue an application experiences.

Therefore, we chose to implement Transaction Chains, a mechanism that leads to serializable transactions with low latency [1]. This mechanism works by splitting transactions into smaller transactions that take place at a single geo-location, defined as a hop. By splitting transactions into hops, transaction chains emulate a serial ordering of transactions. This and some analysis on if hops can run independently ensures serviceability while also reducing latency. Another feature of transaction chains which leads to the most benefit to a user is the return after first hop feature. Transaction chains can choose to return at the first hop since the first hop is usually the most essential and contains the information that the user is after. However, for this to take place, our system needs to ensure that the first hop commits before returning and we need to define that sequential hops must commit eventually if the first hop commits. This is the significant feature of transaction chaining which leads to drastically reduced latency.

For our application, we chose to create an Airbnb style platform that would benefit from a low latency first hop. In this application, users can list their property, book properties on a certain day, and view stats about their profile. From this, we defined a schema that allows the platform to function and defined where hops will be in relation to a user. This is necessary since we want the first hop to be the least latency to fully take advantage of the benefits of transaction chaining. However, we will also discuss why partitioning the hops is challenging. Since the first hop must always commit in a transaction chaining system, the first hop is limited by the transactions that are possible by the system.

Finally, we will discuss the performance of such a system compared to a system that uses previous generation serialization techniques.

2 Background

2.1 Transaction chains

A chain encodes transaction T into several hops. Each hop can be executed on different servers. A hop may have an input from the previous hop. There are four characteristics of hops. First, each hop is isolation. Second, hops are executed in the order, hop p_{i+1} will never be executed before p_i . Third, all-or-nothing atomicity, which the ACID properties of transactions, is guaranteed. Forth, origin ordering is guaranteed. If two transactions are executed in the same server, when transaction the first operation of T_1 executed before the one of T_2 , then every operation p_i in T_1 is executed before every operation p_i in T_2 if they are executed in the same server, which means that concurrent execution may occur. Transaction Chopping ensure the serializability of chains. However, transaction chains has two constraints. First, aborts caused by application can only be done before the first hop. Second, chains are static, which means that operations in transactions are known before execution.

Transaction chain can be connect together for execution.

2.2 Transaction Chopping

Transaction Chopping is the method to divide a large transaction into smaller pieces. The analysis of the transaction chain is based on transaction chopping. Transaction chopping use the smaller pieces to make a graph, SC-graph, showed the relationship between the small pieces. In SC-graph, we have vertices and edges. Vertices mean the small pieces of transactions. Edges mean the relationship. There are two types of edges, S-edges and C-edges. S-edges connect vertices of the small pieces in the same transactions. C-edges connect vertices of the small pieces which are in different transactions but access the same data and an access is a write. An SC-cycle is a cycle contains an S-edge and a C-edge. If the execution doesn't contain an SC-cycle, then the transaction is serializable.

3 Application Design

3.1 Schema Design

To achieve the necessary functionality of our bookings application, we chose three tables: Users, Availability, Properties. The Availability table contains the list of available days a property may be available. That is, if a User would like to list a property on December 12, 2024, they would need to insert into the Availability table with the id of the property and the day it is available. The Users table simply stores information about that user (such as name). The Properties table stores the properties that are listed on the application and some information about that property. Note, that our application does not consider the price of a booking (however, this can easily be added). The database schema design is shown below:

We can see that in Figure 1, the user table contains information about the user, such as name, bookings made, and properties count. The bookings made field contains a count of how many bookings that particular user made. The properties count field keeps track of how many properties a user has listed on the platform. The Availability table contains the foreign keys property id (the id of the property associated with that availability) and booked by which describes which user has booked the property on that day. It also contains a date and booked field to indicate the date of the booking and if it is booked or not.

3.2 Transaction Design

We can see that in Figure 2, there are four transactions in the applications.

T_1 is a transaction to book property. The vertice $T_{1.1}$ is a read operation. If booked, the transaction read the booked from Availability table. The vertices $T_{1.2}$ and $T_{1.3}$ are the write operation, the transaction updates the corresponding data. $T_{1.2}$ writes booked and $T_{1.3}$ writes booked_by in Availability table. The vertice $T_{1.4}$ and $T_{1.5}$ are to increment bookings made in user's side. $T_{1.4}$ reads bookings_made and

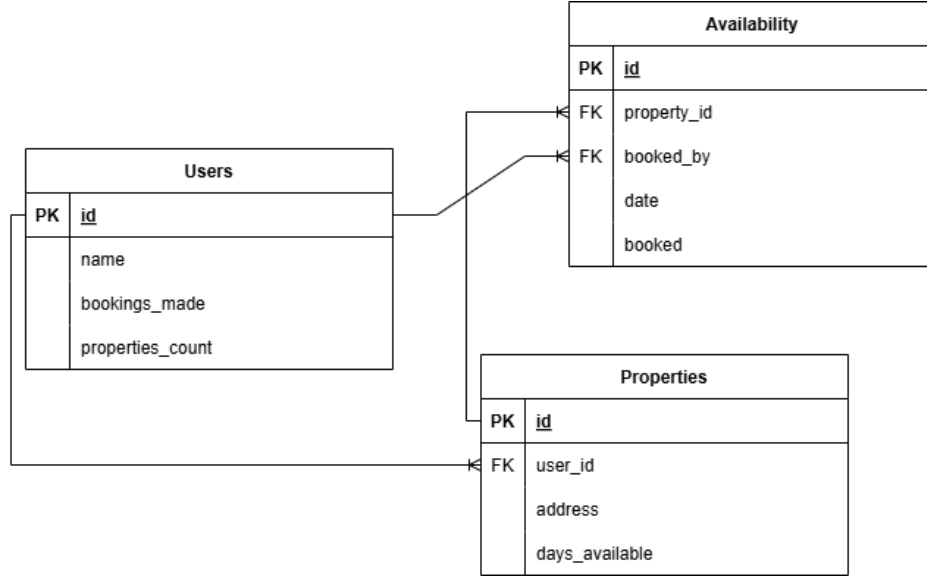


Figure 1: Table Schema

$T_{1.5}$ writes bookings_made in User Table.

T2 is a transaction to update property days available. The vertex $T_{2.1}$ is a read operation to count(read) in Availability Table where booked=false and property_id=property. $T_{2.2}$ is a write operation to update days_available in Property table.

T3 is a transaction to update properties count. $T_{3.1}$ is a read operation to count properties where user_id=user. $T_{3.2}$ is a write operation to update properties_count in User Table.

T4 is a transaction to create Property. $T_{4.1}$ is a read operation to check if property limit reached properties_count ; 5. $T_{4.2}$ is a write operation to insert the property into Property Table.

$T_{1.2}$ and $T_{2.1}$ are different transactions and they both access booked in Availability table and one of them try to write on it. So there is a C-edge between the two vertices. $T_{2.2}$ and $T_{3.1}$ both access days_available in Property table and one of them try to write on it. A C-edge need to be created between them. $T_{3.2}$ and $T_{4.1}$ both access properties_count in User Table and one of them is a write operation, so they have a C-edge.

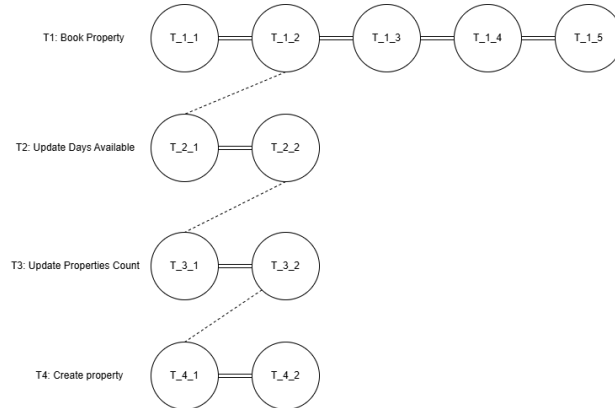


Figure 2: SC-graph

The SC-graph of the transaction we implement is Figure 2. Double lines are S-edges. Dotted lines are C-edges. Because we can't find a cycle inside this figure. The transactions we implement are serializable.

4 System Design

4.1 Transaction Chaining Implementation

To implement transaction chains in our bookings application, we chose to use Python. This choice is based on the popularity of Python as a backend system in many web applications. In particular, we chose to use Flask to host nodes on our machine at different ports. This choice was necessary in our situation since we were limited to a single machine. However, to emulate different hop lengths, we included an added latency at each hop based on their "distance" from the user. At each hop, a Flask server was used to interact a SQL based database. We chose to use sqlite3 as our database system since it is lightweight and can be stored on a single file. Each hop is represented as a class and denoted by an HTTP endpoint. For example, the transaction represented by checking if a booking is available and setting booked to true would define the following method in the Availability hop:

```
class Availability:
    def __init__(self):
        ....
    def check_availability(self, date, property_id):
        self.cur.execute("""
            SELECT booked FROM Availability WHERE date = ? AND property_id = ?
        """, (date, property_id))
        return self.cur.fetchone()[0]
    def book_property(self, date, property_id):
        self.cur.execute("""
            UPDATE Availability SET booked = TRUE WHERE date = ? AND property_id = ?
        """, (date, property_id))
        self.con.commit()

@app.route('/book-property', methods=['POST'])
def add_booking():
    data = request.json
    date = data['date']
    property_id = data['property_id']

    availability = Availability()
    if availability.check_availability(date, property_id):
        return jsonify({'status': 'error', 'message': 'Property already booked'})
    availability.book_property(date, property_id)

    return jsonify({'status': 'success'})
```

Note that since the book-property endpoint can abort (checking if a property is booked) it must be the first hop. This is one of the limitations of our system which controls the latency a user might see. However, we can control this by partitioning the data to follow the most popular transactions of our system. This also notes how we represented pre-defined methods at each hop.

To manage and execute or chains, we also defined a chain handler. This handler is the bulk of the transaction

chaining mechanism. It works by using an add hop method. The chain is represented by a simple python list. The implementation of the chain and add hop method is shown below:

```
class Chain:
    def __init__(self):
        self.chain = []

    def add_hop(self, url, args=None, prev_return_as_arg=False):
        self.chain.append({
            "url": url,
            "args": args if args else {},
            "prev_return_as_arg": prev_return_as_arg
        })
```

The add hop contains a url, args, prev_return_as_arg fields. The url field describes the method and the hop to be executed. For example, in the book property transaction, the url may be `http://localhost:5000/book-property` indicating that the availability table is at the hop at port 5000 and we want to run the book-property method. The args field contains the arguments that need to be sent to the hop. For example, property id and date would be contained in the args field in the make booking transaction. Finally, the prev_return_as_arg field describes whether or not the hop depends on a value from the previous hop. In cases where a hop depends on a previous hop's value, the system needs to wait for that value to be returned before executing sequential hops.

The last item contained within our chain manager is the execute method which executes the hops in the transaction chain. To make the most of transaction chaining, we chose to run hops asynchronously (in parallel). For this, we made use of Python's asyncio and aiohttp. This allowed us to run hops asynchronously when they can be. However, note that not in every case can hops be run asynchronously. Our implementation can be seen below:

```
async def execute(self, session):
    prev_result = {}

    first = True

    for hop in self.chain:
        url = hop['url']
        args = hop['args'].copy()

        if hop['prev_return_as_arg']:
            args.update(prev_result)

        try:
            if hop['prev_return_as_arg'] or first:
                first = False
                async with session.post(url, json=args) as response:
                    if response.status != 200: #ABORT at first hop
                        raise ValueError(...)
                    response_data = await response.json()
                    prev_result = response_data.get('result', {})
            else:
                asyncio.create_task(self._send_request(session, url, args))

        except Exception as e:
```

```

    print(f"Error during chain execution: {e}")
    return None

return prev_result

```

We can see that the chain manager waits for the return of the first hop before firing the remaining hops. Additionally, if a hop depends on a value passed from a previous hop, it waits until the return is received.

4.2 Dynamic Transaction Handling

To handle dynamic transactions, we chose to avoid the analysis of the sc-graph entirely (since the overhead will likely take longer than the average latency). Instead, we chose to implement Two-Phase Locking to ensure that transactions are still serializable. Each hop (Flask server) holds a Lock Table. When a specific data object is accessed (read or write), that transaction obtains a lock (either shared for reads, or exclusive for writes). This implementation is simple in our application since we have a small number of predefined fields for each table. However, when does a row wise delete/insert, the transaction must hold a lock on the entire table, which could lead to lower performance. To handle deadlocks, however, we implemented a wait-die strategy using timestamps. Older transactions will have an older timestamp and is able to wait for locks held by newer transactions. On the other hand, if a newer transaction attempts to obtain a lock held by an older transaction and no progress is made (timeout), the newer transactions aborts itself. This ensures that dynamic transactions are still serializable. However, this means that dynamic transactions do not benefit from the performance gained from transaction chaining.

5 Evaluation

5.1 Performance

To evaluate the performance of transaction chains in our application, we chose to compare serial transaction execution and transaction chained execution. We considered our 4 standard transactions, and executed a different number of transactions at once. We recorded the average latency experienced by each transaction (chain) as well as the latency of the first hop (the return). We also measured the latency observed by a system that implement no serializability (no transaction chains). The results are shown below in Table 1 and 2:

Transactions	Serial Latency (ms)	Transct. Chain Avg Latency (ms)	First-Hop Avg (ms)
4	885.6	326.20	157.46
16	3,683.6	356.12	156.05
64	14,836.8	551.23	209.04
256	64,697.6	1,898.44	1,378.74

Table 1: Comparison of Serial Processing and Transaction Chaining

Transactions	Avg Latency (no chain) (ms)
4	221.4
16	230.2
64	231.8
256	252.7

Table 2: Average Latency of Asynchronous Execution Without Chaining

We can see that the average latency experienced by a transaction-chained system increased drastically at 256 transactions (likely reached the max the server can handle). However, we see that the transaction

chained system drastically reduced the total latency to execute a transaction compared to the serial execution (with no parallel execution). We also observe that the first hop latency (the latency a user perceives) is incredibly fast. Even in the system with no chaining and an asynchronous execution, the first hop latency was drastically reduced (until 256 transactions). Transaction chaining does add an overhead to the latency however compared to the asynchronous execution with no chaining. However, this small overhead is worth paying to guarantee serializability. The system with no chaining does not guarantee anything about the execution of transactions.

5.2 Dynamic Transaction Performance

Since our system uses standard Two-Phase Locking (which can block transactions), we expect to observe a significant decrease in performance when handling dynamic transactions. To test the performance of the 2PL layer of our application, we decided to use the same transactions (even though they are SC-Cycle free) and introduced some new transactions, such as deleting a booking, updating a property, and updating an availability. We observed the performance of the 2PL when sent 4, 16, 64, and 256 concurrent transactions. The results are shown below in Table 3:

Transactions	Avg Latency (ms)
4	259.6
16	487.3
64	1,977.0
256	5,087.4

Table 3: Average Latency using 2PL

As the number of concurrent transactions grew, we see an incredible rise in latency when using 2PL. This is because as the number of transactions increased, so do the number of conflicting transactions. Thus, some transactions are blocked and some even need to abort and restart. However, we found that the overhead for 2PL is small when using a small number of transactions (4) which led to a fast execution of each transaction.

5.3 Transaction Ordering and Limitations

Note that transaction chains do not guarantee a total ordering of transactions [1]. We will show an example of this where the output of two transactions can differ based on the somewhat random ordering performed by the machine. In this example, we initially have two users, Alice and Bob. Alice creates a property and makes it available on two separate days (January 1, 2025 and January 2, 2025). Alice books the January 1st booking. Therefore we construct two transactions: one where Alice books the January 1st booking and one that updates the days a property is available. Since both these transactions run asynchronously, any order can occur with transaction chaining. In fact, we observed that T2, T1 occurred in practice even though T1 is defined before T2 (asynch). Below, in Figure 3, we observe the output of such a transaction sequence:

	id	property_id	price	user_id	date	booked
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	100.0	1	2025-01-01	1
2	2	1	300.0	1	2025-01-02	0

	id	address	user_id	days_available
	Filter	Filter	Filter	Filter
1	1	123 Main St	1	2
2	2	456 Elm St	2	0

	id	name	properties_count	bookings_made
	Filter	Filter	Filter	Filter
1	1	Alice	0	1
2	2	Bob	0	0

Figure 3: Tables After Executing Two Transactions

Figure 3 above shows that `days.available` is updated to 2 even though one of the days was booked by Alice (indicating that the read of T2 happened before T1’s write). This is one of the limitations of transaction chaining. Another limitation of transaction chaining is the type of transactions that can occur in the system. We are limited to only acyclic SC-Graph transactions. However, in practice many transactions do create an SC-Cycle which is why it is a limitation of transaction chaining.

6 Conclusion

6.1 Summary

Transaction Chains is implemented on the Airbnb application. Transaction Chains is to split transactions into smaller pieces called hops. Transaction Chains is based on transaction chopping, which creates SC-graph and ensure serializability of transactions. The application accesses three tables, Availability, Users and Properties with four transactions. To process dynamic transactions, 2PL is implemented and also guarantees the serializability. The application sees a significant increase in performance when utilizing transaction chains. In particular, the perceived latency by a user is incredibly fast since the first hop in our application was quick. However, when handling dynamic transactions, since we used 2PL, we see a lack of performance and high latency.

6.2 Future Work

We would have liked to implement the Static Analysis portion of transaction chaining [1]. Although, we handled dynamic transactions, to guarantee atomicity, Two Phase Commit should also be implemented in addition to 2PL. This would have guaranteed the atomicity of the system (we assumed the system to be atomic for dynamic transactions for analysis purposes). Given that static analysis would have avoided running 2PL and 2PC, we would have likely seen a performance increase for most transactions since 2PL is dependent on the conflict probability [2].

References

- [1] Zhang, Y., Power, R., Zhou, S., Sovran, Y., Aguilera, M. K., & Li, J. (2013). Transaction chains: achieving serializability with low latency in geo-distributed storage systems. *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*, 276–291.
- [2] Thanos, C., Bertino, E., & Carlesi, C. (1988). The effects of two-phase locking on the performance of a distributed database management system. *Performance Evaluation*, 8(2), 129–157.
- [3] Tomsic, A. Z., Bravo, M., & Shapiro, M. (2018). Distributed transactional reads: the strong, the quick, the fresh & the impossible. *Proceedings of the 19th International Middleware Conference*, 120–133.